

fsb

A read-only file browser in your web browser, and how it is built

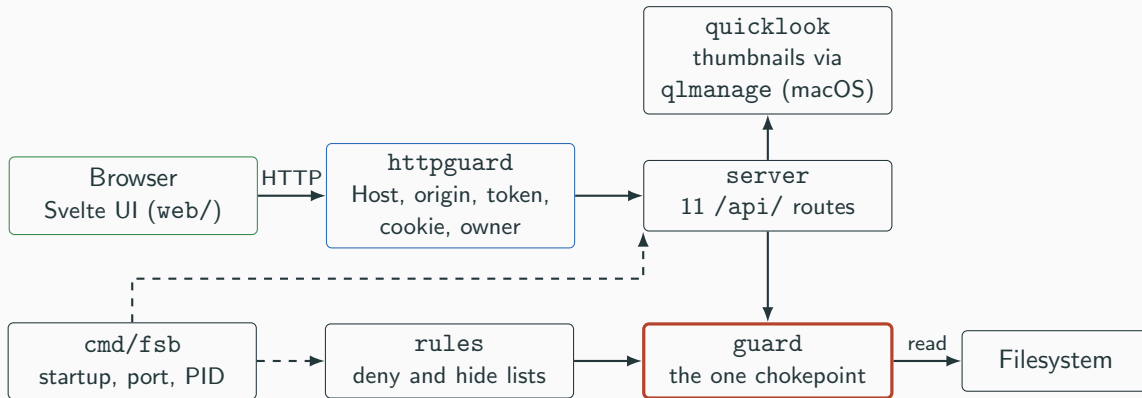
October 2026 · describes fsb 1.0.12

What fsb is

- One Go binary that serves a browser UI on 127.0.0.1
- Browse, sort, filter, search and preview your own files, on macOS or Linux
- **Read-only by construction:** only GET endpoints, no code that writes your files
- **Local-only:** loopback address, Host and Origin checks, a single-use launch token
- **Deny rules** (`~/.ssh`, keychains, browser profiles, ...) refused on every path, answered as “not found”

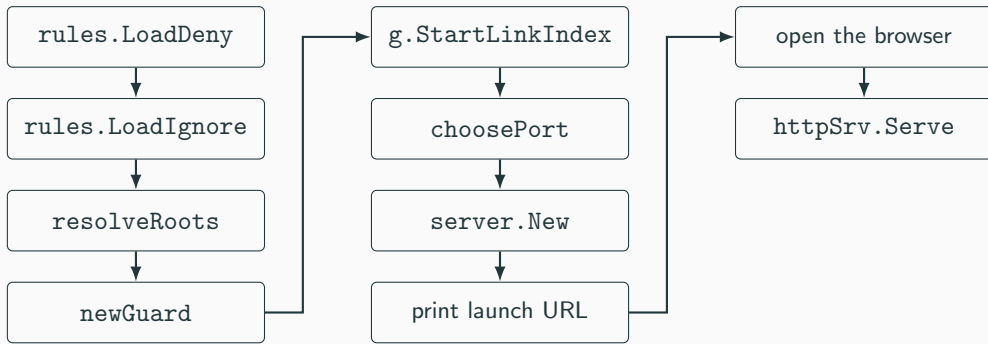
The promise is that it is safe to point at a real filesystem. Most of the code exists to keep that promise.

Architecture



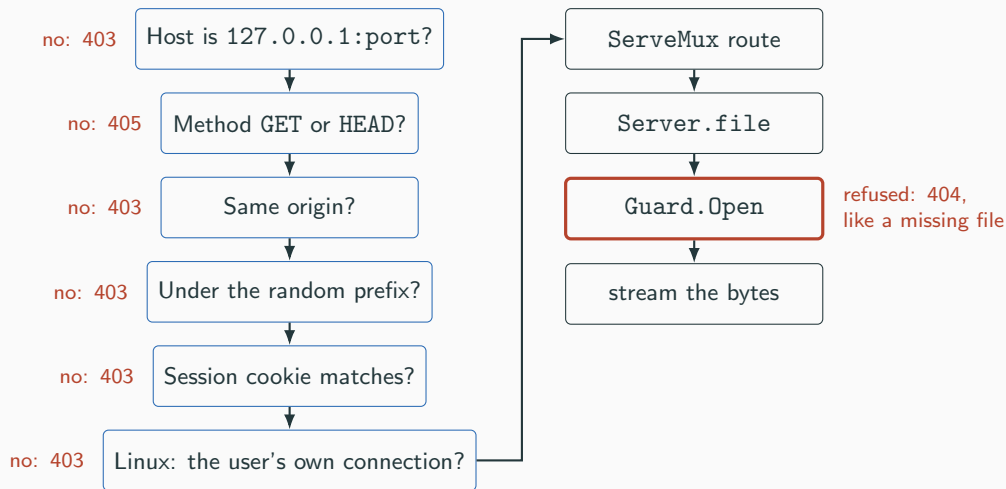
Every file access goes through guard; no handler opens a path itself.

Startup: `cmd/fsb run()`



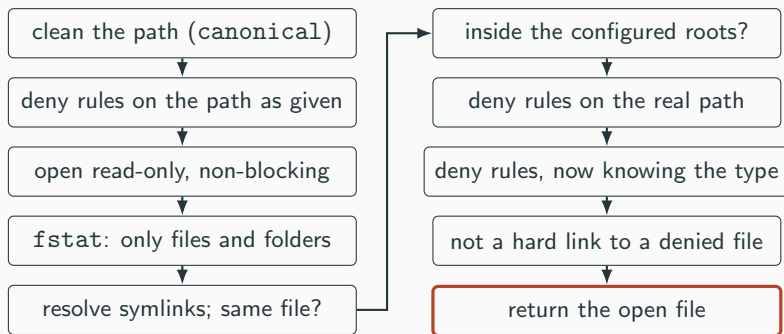
- Core deny rules are built in; a missing one raises a warning banner
- The hard-link index builds in the background; until it is ready, files with several names are refused
- The port is on `127.0.0.1` only; the interface cannot be configured

One request: GET /prefix/api/file?path=...



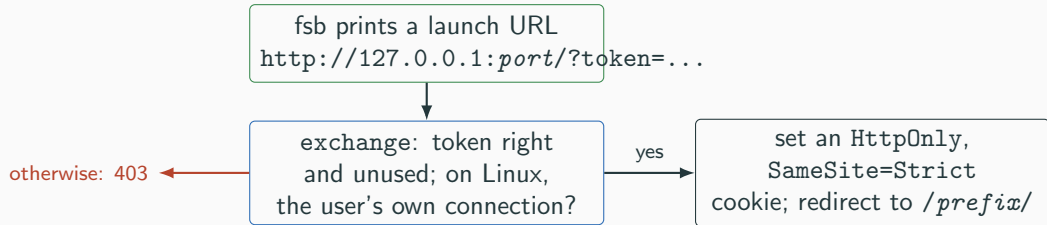
Left column: `httpguard.Middleware`, the same for every route. On Linux the owner of the connection is looked up once per connection; macOS has no such check.

The chokepoint: `guard.open`

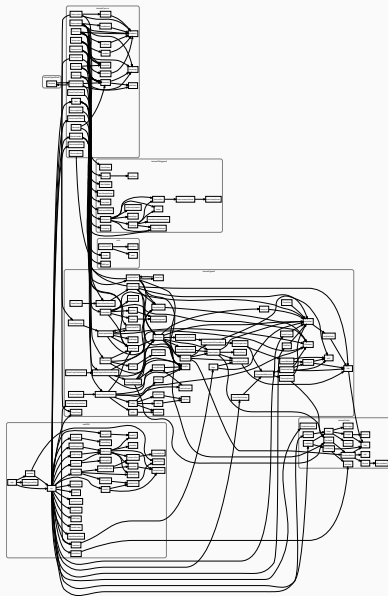


- Opening first and then checking what was opened closes the race between check and use
- FIFOs, sockets and devices are rejected before anything is read

Logging in without a password

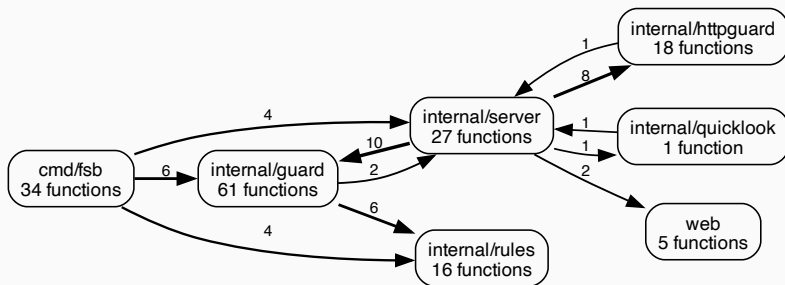


- The token works once (an atomic compare-and-swap), then leaves the address bar
- The prefix is random and not in the launch URL: a user who reads the opener's command line learns only a spent token
- Other paths are refused like unauthenticated requests; comparisons are constant-time
- Linux: another user's connection is refused, at launch and on every request, without using up the token



Every call between fsb's own functions: 162 functions, 251 calls. Generated from the code, not drawn.

Calls between packages



The same graph folded by package; numbers are distinct caller–callee pairs. The arrows back into server are callbacks: server hands guard and quicklook a function to call for each result.

Where the functions are

Package	Functions	Role
internal/guard	61	the chokepoint: open, list, search, archives, links
cmd/fsb	34	startup, port choice, background mode, opener
internal/server	27	HTTP handlers
internal/rules	16	gitignore-style rule matching
internal/httpguard	18	localhost protections, login, connection owner
web	5	embedded UI, Markdown frame
internal/quicklook	1	thumbnails for Office and iWork

Counted from the call graph: functions that call or are called by another fsb function.

By the numbers

- About 4,000 lines of Go
- About 8,100 lines of Go tests
- About 3,100 lines of Svelte and TypeScript
- Browser tests in Chrome and Safari, against macOS and four Linux distributions
- About 140 commits since 20 September 2026
- Functional, design and algebraic specs in $\text{\LaTeX}$
- Independent reviews at each new boundary
- Source only; provided as-is

How these slides were made

1. **Call graph** from Go's own analysis (golang.org/x/tools):
`callgraph -algo=vta ./cmd/fsb`
2. **Filter**: a short Python script keeps calls between fsb's own functions and writes a Graphviz file; dot draws it
3. **Diagrams**: drawn by hand in TikZ, checked against the graph
4. **Slides**: $\text{\LaTeX}$ Beamer, theme *moloch*

`make` in `docs/talk/` rebuilds everything from the current code.